Evaluating Checkpoint/Restore-Based Pod Migration in Kubernetes: An Empirical Performance and Bottleneck Study

Musa Khasmammadov
Independent Systems Researcher
Baku, Azerbaijan
musaxasmammedov77@gmail.com

Abstract—Stateful singleton containers lacking distributed consensus lose volatile heap state during Kubernetes node maintenance and cloud Spot instance preemptions. We evaluate an automated, snapshot-consistent pod migration pipeline coordinating CRIU (v4.3), the Kubelet Checkpoint API (KEP-2008), Buildah OCI packaging, CRI-O (v1.31), and Cilium eBPF routing across AWS EC2 nodes. Across 36 decoupled migrations compared against 20 native cold reschedules of a stateful service, median cutover blackout is 1064.5 ms versus 1019.0 ms (+4.5%), while sample means show an overhead of 125.4 ms (+12.5%, 1132.4±241.1 ms vs. 1007.0±231.2 ms, 95% CI: $[-6.7, +257.4]$ ms; Hodges-Lehmann difference: 136.0 ms, 95% CI: $[-18.0, +252.0]$ ms; Mann-Whitney $U = 252.5, p = 0.0660$). External 50 Hz probing yields median client blackouts of 1080.0 ms vs. 1040.0 ms (systematic +15.5– + 21.0 ms offset from discrete 20 ms sampling; external mean difference: 122.7 ms, closely mirroring the 125.4 ms operator difference), with variance dominated by Kubelet's 1 s PLEG relist cycle. Decoupled migration incurs bounded state drift of 669.0 ms under default packaging (560.0 ms in an exploratory uncompressed trial), whereas an analytical stop-and-copy projection achieves zero drift by extending blackout to 1.88 s (49.1 s at 1 GB under -D). In exploratory scaling up to 1024 MB (1 GB), uncompressed Buildah packaging slashes build time from 32.06 s to 13.74 s (2.3×), completing source evacuation in 36.78 s within the 120 s Spot deadline and verifying 100% SHA-256 bit-level integrity, while observing writeback throughput (137.9 MiB/s) consistent with EBS gp3 baseline storage limits (125 MiB/s). Compared to analytical cold state reconstruction from cloud storage (11–30 s), checkpointing eliminates external persistence tiers, trading bounded state drift (0.56–3.16 s under -D vs. 0.67–7.48 s under gzip) for 1.77–5.64 s stop-and-copy cutover at $\leq$ 256 MB under -D (1.88–9.96 s under gzip). Finally, single-container KEP-2008 checkpointing severs in-pod loopback sockets, favoring node-level eBPF routing.

Index Terms—Kubernetes, Container Checkpoint/Restore, CRIU, CRI-O, Cilium eBPF, Stateful Workloads, Spot Instances, Recovery Point Objective.

I. Introduction

Kubernetes manages workloads under the foundational assumption that container instances are ephemeral and stateless [1]. When worker nodes undergo kernel maintenance (kubectl drain) or cloud virtual machines are reclaimed, stateless microservices maintain availability by spinning up replacement replicas before terminating old instances (maxSurge).

However, critical production classes maintain large, mutable in-memory heaps without built-in distributed consensus:

1) Scientific & Numerical Solvers: Iterative physics simulations, linear algebra solvers, and legacy C++/Python runtimes accumulating state over hours.
2) Warm ML Inference Caches & Aggregators: High-dimensional embedding buffers, key-value transformer caches, and session aggregators where rebuilding state from cold storage imposes severe warm-up latency.

Deploying these workloads on Amazon Web Services (AWS) EC2 Spot instances yields 70–90% compute cost savings [2], [17], but subjects pods to abrupt reclamation with a 120-second termination notice [3]. Standard eviction issues SIGTERM followed by SIGKILL, destroying volatile RAM. Offloading state to external tiers such as Redis [4] prevents data loss, but incurs intrusive architectural refactoring and replaces nanosecond memory dereferences with network roundtrips.

Userspace checkpoint/restore via CRIU [5] provides an application-transparent alternative. In hypervisor literature, "live migration" classically refers to iterative pre-copy schemes [10] that track dirty pages via hardware MMU page tables, streaming memory while the virtual machine executes to achieve millisecond cutovers. In container environments lacking hardware virtualization traps, process-level live migration via CRIU and Kubernetes KEP-2008 [8] is operationally distinct. In this paper, we use the term checkpoint/restore-based container migration to denote cross-node state transfer that preserves running process heaps and listening sockets without re-executing main() or rebuilding application memory structures, while accepting a sub-second cutover blackout bounded by snapshot serialization and container runtime event loops.

This paper makes four empirical systems contributions:

- End-to-End Controller: Privileged DaemonSet operator (PodLiveMigration CRD) automating KEP-2008 checkpointing, Buildah packaging, registry transfer, CRI-O restore, and Cilium eBPF routing across AWS

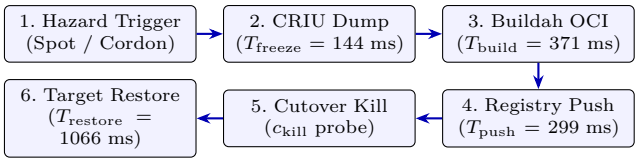


Fig. 1. Decoupled snapshot-consistent migration pipeline across nodes.

EC2.

- Statistical Evaluation & Runtime Profiling ($N = 36$ CRIU vs. $N = 20$ Cold): Under 50 Hz probing, inferential tests ($p = 0.0660$, Hodges-Lehmann: 136.0 ms; Welch's $t = 1.91, p = 0.0628$) and tick regression ($r = 0.9624$) show blackout is dominated by Kubelet PLEG jitter. Direct daemon profiling confirms container unfreezing occurs at 478–516 ms; we hypothesize that downstream Kubelet status reporting and EndpointSlice propagation gate end-to-end client cutover.
- 1 GB Scaling & Packaging Optimization: In exploratory scaling up to 1024 MB ($N = 1$), 100% SHA-256 bit integrity is verified. We observe writeback throughput (137.9 MiB/s) consistent with EBS gp3 baseline storage limits (125 MiB/s [18]) at 1 GB, and show uncompressed Buildah packaging slashes build time from 32.06 s to 13.74 s (2.3×, completing evacuation in 36.78 s within the 120 s Spot deadline).
- State Reconstruction Benefit & Sidecars: We contrast checkpointing against analytical cold state reconstruction (11–30 s for 1 GB), analyze RPO trade-offs, and profile single-container KEP-2008 sidecar loopback constraints.

II. System Architecture

A. Pipeline Design and CRIU Configuration

Our testbed runs on Ubuntu 24.04 LTS (Linux 6.8) with CRIU v4.3 [5], Kubelet v1.31.14 (KEP-2008 enabled) [8], CRI-O v1.31.4 with the runc runtime, Buildah, and Cilium v1.16.3 (kubeProxyReplacement=true) [9]. Figure 1 details the six-stage migration pipeline.

Because a restored pod receives a new Pod IP from the target node's CIDR (10.244.2.0/24 → 10.244.1.0/24), /etc/criu/default.conf specifies key flags [5]:

- tcp-close: Restores listening sockets (0.0.0.0:8090) in the LISTEN state while closing established TCP sockets bound to the old Pod IP.
- skip-in-flight: Drops transient in-flight TCP handshakes occurring during 50 Hz probing at the instant of ptrace seizure.
- skip-mnt /run/.containerenv: Skips CRI-O's transient metadata mount so CRImportCheckpoint mount validation succeeds.
- ext-unix-sk and shell-job: Permits dumping external UNIX domain sockets and inherited stdio descriptors.

B. Cutover Blackout Definition

Our benchmark service uses HTTP/1.0 behind a Kubernetes Service ClusterIP and Cilium L7 Envoy Ingress without long-hold request buffering. Between source pod deletion (t_{kill}) and target endpoint readiness (t_4), incoming requests fail immediately with connection refused or HTTP 503. We define cutover blackout T_{blackout} strictly as the unavailability window during which probes fail, measured both by the operator clock ($t_4 - t_{\text{kill}}$) and by an external 50 Hz (±20 ms) prober as the maximum interval between consecutive HTTP 200 OK responses.

III. Latency, State Drift, and Evacuation Model

A. Stage Timestamps and Disruption Decomposition

The operator records monotonic timestamps around each pipeline step: t_0 (pre-checkpoint), t_1 (post-dump), t_2 (post-build), t_3 (post-push), t_{kill} (pre-delete probe c_{kill} and pod deletion), and t_4 (first HTTP 200 OK from the restored pod):

$$T_{\text{freeze}} = t_1 - t_0, \quad T_{\text{build}} = t_2 - t_1, \quad T_{\text{push}} = t_3 - t_2 \tag{1}$$

$$T_{\text{restore}} = t_4 - t_3, \quad T_{\text{blackout}} = t_4 - t_{\text{kill}} = T_{\text{restore}} - t_{\text{probe}} \tag{2}$$

where $t_{\text{probe}} = t_{\text{kill}} - t_3 \approx 1$–$2$ ms is pre-kill probe latency. Client-perceived disruption consists of source freeze stutter T_{freeze} (threads paused by ptrace) plus cutover blackout T_{blackout}:

$$T_{\text{disruption}} = T_{\text{freeze}} + T_{\text{blackout}} \tag{3}$$

$$T_{\text{total}} = T_{\text{freeze}} + T_{\text{build}} + T_{\text{push}} + T_{\text{restore}} \tag{4}$$

B. State Drift (RPO), Post-Restore Ticks, and Rollback

In Mode A (Decoupled, KeepRunning: true), CRI-O unfreezes the source container at t_1, allowing it to serve traffic while Buildah packages and pushes the image. State mutations occurring between the snapshot ($S_{\text{ckpt}} \approx c_{\text{dump}} + 1$) and pod deletion ($t_{\text{kill}}$) are lost upon cutover. At mutation rate λ, Recovery Point Objective (RPO) and lost updates ΔS_{lost} are:

$$\text{RPO} = T_{\text{build}} + T_{\text{push}} \tag{5}$$

$$\Delta S_{\text{lost}} = c_{\text{kill}} - S_{\text{ckpt}} \approx \lambda \cdot (T_{\text{build}} + T_{\text{push}}) \tag{6}$$

Workloads with external side effects require idempotency keys or write-ahead logging (WAL).

In Mode B (Synchronous Stop-and-Copy, Analytical Projection), halting the source container at t_1 yields RPO ≡ 0, but extends service unavailability to the entire pipeline ($T_{\text{blackout}}^{\text{stop-and-copy}} = T_{\text{total}}$).

On the target node, runc restore unfreezes the process at $T_{\text{unfreeze}} < T_{\text{restore}}$, and the restored process immediately resumes incrementing from S_{ckpt}. However, the operator only probes c_{restore} after Kubelet's PLEG loop (1 s period) updates Pod.status.phase = Running at $T_{\text{restore}} = T_{\text{unfreeze}} + \Delta t_{\text{pleg}}$:

$$c_{\text{restore}} - c_{\text{dump}} \approx \lambda \cdot (T_{\text{restore}} - T_{\text{unfreeze}}) \tag{7}$$

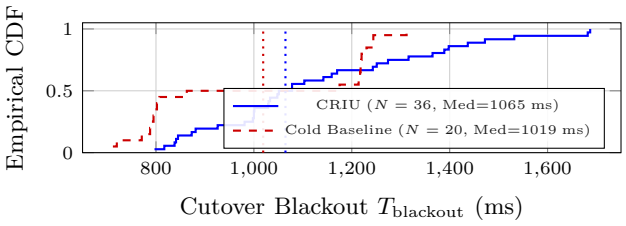


Fig. 2. Empirical CDF of Cutover Blackout (T_{blackout}). Dotted lines mark sample medians (CRIU: 1064.5 ms, Cold Baseline: 1019.0 ms).

C. Source-Side Spot Evacuation Condition

When AWS issues a 120-second Spot Interruption Notice, only the source node is terminated at $t = 120$ s; target-side pull and restoration execute on the surviving destination node after t_3. Spot survival requires only that worst-case source evacuation $T_{\text{evac}}^{\text{worst}}(M)$ complete within 120 s:

$$T_{\text{evac}}^{\text{worst}}(M) = T_{\text{detect}}^{\text{worst}} + T_{\text{freeze}}(M) + T_{\text{build}}(M) + T_{\text{push}}(M) < 120 \text{ s} \quad (8)$$

where $T_{\text{detect}}^{\text{worst}} = 5.0$ s accounts for the worst-case detection latency under a 5 s IMDSv2 polling period.

IV. Empirical Evaluation

A. Testbed Setup

Experiments were conducted on a three-node AWS EC2 cluster in eu-central-1: one c7i-flex.large control plane (2 vCPU, 4 GB RAM, hosting the OCI registry at 10.10.1.10:5000 and Cilium Ingress) and two m7i-flex.large workers (2 vCPU, 8 GB RAM, EBS gp3 storage). All 11 microservices of Google Online Boutique ran continuously to generate background multi-tenant CPU, memory, and CNI contention.

B. Experiment 1: Statistical Evaluation ($N = 36$ CRIU vs. $N = 20$ Cold Baseline)

Table I summarizes measurements across $N = 36$ CRIU migrations ($n = 20$ w1 $\rightarrow$ w2; $n = 16$ w2 $\rightarrow$ w1) and $N = 20$ native cold reschedules of the 26 MB stateful service ($\lambda = 10$ Hz). Table II presents individual trace breakdowns and the Mode B projection. Figure 2 displays the empirical cumulative distribution function (ECDF) of cutover blackout.

Three crucial findings emerge:

1) Cutover Blackout and Prober Validation: Median operator blackout is 1064.5 ms for CRIU vs. 1019.0 ms for cold restart (+4.5%), while means show an overhead of 125.4 ms (+12.5%, 1132.4 ± 241.1 ms vs. 1007.0 ± 231.2 ms, 95% CI $[-6.7, +257.4]$ ms; Hodges-Lehmann diff: 136.0 ms, 95% CI $[-18.0, +252.0]$ ms; Mann-Whitney $U = 252.5, p = 0.0660$; Welch's $t(40.8) = 1.91, p = 0.0628$). External 50 Hz probing yields medians of 1080.0 ms vs. 1040.0 ms: this systematic $+15.5 - +21.0$ ms offset is directly explained by discrete 20 ms sampling quantization ($\Delta\tau_{\text{poll}} = 20$ ms). External means differ by 122.7 ms (1148.2 vs. 1025.5 ms; $U = 259.0, p = 0.078$), closely mirroring the 125.4 ms operator difference. Because Kubelet PLEG relisting introduces common variance ($\sigma \approx 230$–240 ms), T_{blackout} measures Kubelet loop scheduling rather than container re-instantiation.
2) PLEG Dominance and Container Runtime Datapath: Standard deviations for CRIU ($\sigma = 241.1$ ms) and cold restart ($\sigma = 231.2$ ms) match uniform 1 s PLEG relist jitter ($1000/\sqrt{12} \approx 288.7$ ms). Tick regression ($r = 0.9624$) indicates unfreezing at $T_{\text{unfreeze}} = 537.1$ ms, which direct CRI-O daemon profiling confirms at 478–516 ms (journalctl -u crio: StartContainer: ≈ 310 ms pull, ≈ 168 ms restore). Client traffic traverses ClusterIP and Cilium Ingress: because the operator awaits Pod.status.phase = Running before probing, and the external prober queries Service IP, we hypothesize that the residual 261–1151 ms interval reflects Kubelet PLEG relisting and downstream EndpointSlice convergence rather than container instantiation.
3) Trace Dynamics and Methodological Threats: In Trace #35 (51 $\rightarrow$ 60 $\rightarrow$ 57), 5 post-restore ticks accumulated during 543 ms of PLEG delay, whereas Trace #36 (77 $\rightarrow$ 85 $\rightarrow$ 79) experienced only 1 tick. Runs were executed in directional series ($n = 20$ w1$\rightarrow$w2, $n = 16$ w2$\rightarrow$w1) with low dirtying rates; non-randomized execution and write-heavy workloads represent threats to validity.

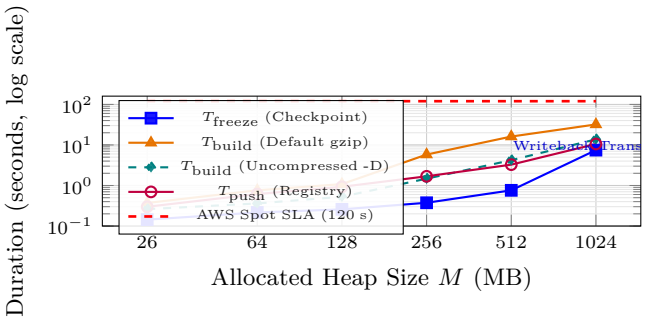


Fig. 3. Empirical heap scaling on AWS EC2 (26–1024 MB) showing EBS gp3 baseline throughput limits and Buildah uncompressed packaging optimization.

C. Experiment 2: Empirical Heap Scaling, SHA-256 Verification, and Gzip Optimization

We populated a contiguous heap buffer of $M \in \{64, 128, 256, 512, 1024\}$ MB with high-entropy pseudo-random float64 weights (simulating an ML embedding table or scientific grid). Prior to checkpointing on k8s-worker-1, the service computed the SHA-256 digest over the entire heap; upon restoration on k8s-worker-2, the /live/verify endpoint recomputed and validated the digest. Table III and Figure 3 present the measurements.

Three systems insights emerge:

TABLE I
Statistical Evaluation: $N = 36$ CRIU Decoupled Pod Migrations vs. $N = 20$ Native Cold Reschedules (26 MB Stateful Service)

Configuration / Subset	T_{freeze} (ms)	T_{build} (ms)	T_{push} (ms)	RPO (ms)	T_{restore} (ms)	T_{blackout} (ms)	$T_{\text{disruption}}$ (ms)	T_{total} (ms)	$c_{\text{kill}} - c_{\text{dump}}$ (ticks)
CRIU w1 → w2 ($n = 20$)									
Median ($p50$) / $p95$	144.0 / 158	368.5 / 400	293.0 / 310	664.0 / 703	1033.0 / 1533	1031.5 / 1532	1178.0 / 1673	1857.0 / 2358	7.5 / 8
Mean ± SD [Min–Max]	143.3 ± 9.4	370.9 ± 17.1	293.6 ± 12.5	664.6 ± 27.6	1112.9 ± 249.2	1111.2 ± 249.3 [817–1687]	1254.5 ± 250.1 [958–1836]	1920.8 ± 248.7	7.6 ± 0.6 [7–9]
CRIU w2 → w1 ($n = 16$)									
Median ($p50$) / $p95$	145.0 / 155	372.5 / 384	301.5 / 308	672.5 / 688	1092.0 / 1474	1090.5 / 1472	1229.0 / 1622	1887.0 / 2277	7.0 / 8
Mean ± SD [Min–Max]	145.4 ± 7.1	366.6 ± 16.4	296.6 ± 12.4	663.2 ± 27.9	1160.2 ± 235.9	1158.8 ± 235.7 [797–1682]	1304.1 ± 238.1 [944–1828]	1968.8 ± 238.0	7.4 ± 0.5 [7–8]
CRIU Combined ($N = 36$)									
Median ($p50$) / $p95$	144.0 / 156	370.5 / 389	298.5 / 310	669.0 / 696	1066.0 / 1533	1064.5 / 1532	1203.0 / 1673	1878.0 / 2358	7.0 / 8 (RAM Kept)
Mean ± SD [95% CI]	144.2 ± 8.4	369.0 ± 16.7	294.9 ± 12.4	663.9 ± 27.4	1133.9 ± 241.1	1132.4 ± 241.1 [1053.6, 1211.1]	1276.6 ± 242.6	1942.1 ± 241.8	7.5 ± 0.6 [7–9]
Native Cold Baseline ($N = 20$)									
Median ($p50$) / $p95$	N/A	N/A	N/A	Total Loss	1019.0 / 1244	1019.0 / 1244	1019.0 / 1244	1019.0 / 1244	100% State Lost
Mean ± SD [95% CI]	N/A	N/A	N/A	Total Loss	1007.0 ± 231.2	1007.0 ± 231.2 [905.7, 1108.3]	1007.0 ± 231.2	1007.0 ± 231.2	Reset to 0 [1–6]

Inferential Tests (T_{blackout}): Mann-Whitney $U = 252.5, p = 0.0660$; Hodges-Lehmann median diff = +136.0 ms (95% CI $[-18.0, +252.0]$ ms); Welch's $t(40.8) = 1.91, p = 0.0628$; Mean diff = +125.4 ms (95% CI $[-6.7, +257.4]$ ms). External 50 Hz Prober ($T^{\text{ext}}_{\text{blackout}}$): CRIU median 1080.0 ms (1148.2 ± 242.5 ms) vs. Cold Baseline median 1040.0 ms (1025.5 ± 233.1 ms).

TABLE II
Detailed Trace Breakdown and Operating Mode Trade-offs (RTO vs. RPO)

Mode / Trace ID	Path	T_{total}	T_{freeze}	T_{build}	T_{push}	T_{blackout}	$T_{\text{disruption}}$	RPO	Counter($c_{\text{dump}} \to c_{\text{kill}} \to c_{\text{restore}}$)
Cold Reschedule (UI Trace)	w2 → w1	1289 ms	N/A	N/A	N/A	1289 ms	1289 ms	Total Loss	43544 → 8 (Lost 43536 ticks)
CRIU Decoupled (Trace #35)	w1 → w2	1935 ms	159 ms	385 ms	310 ms	1078 ms	1237 ms	695 ms	$51 \to 60 \to 57$ ($\Delta S_{\text{lost}} \approx 8, \Delta S_{\text{post}} \approx 5$)
CRIU Decoupled (Trace #36)	w2 → w1	1642 ms	147 ms	388 ms	308 ms	797 ms	944 ms	696 ms	$77 \to 85 \to 79$ ($\Delta S_{\text{lost}} \approx 7, \Delta S_{\text{post}} \approx 1$)
Mode B Stop-and-Copy (Model Projection)	Bidirectional	1878 ms	144 ms	370.5 ms	298.5 ms	1878 ms	1878 ms	0 ms	Zero Drift ($T_{\text{blackout}} = T_{\text{total}}$)

TABLE III
Empirical Memory Scaling on AWS EC2 (26 MB to 1024 MB Heap) with Cross-Node SHA-256 Heap Integrity Verification

Allocated Heap	OCI Layer Size	T_{freeze} (Dump)	T_{build} (gzip)	T_{build} (-D)	RPO (gzip / -D)	Worst T_{evac} (gzip / -D)	% 120s SLA	T_{restore}	T_{total} (gzip / -D)	SHA-256 Digest
26 MB ($N = 36$ Med.)	5.16 MiB	0.144 s	0.371 s	0.261 s	0.669 s / 0.560 s	5.81 s / 5.70 s	4.8% / 4.8%	1.066 s	1.88 s / 1.77 s	Verified (Counter)
64 MB (Weights)	69.11 MiB	0.216 s	0.761 s	0.361 s	1.376 s / 0.976 s	6.59 s / 6.19 s	5.5% / 5.2%	0.951 s	2.54 s / 2.14 s	100% Match (7785bfc7)
128 MB (Weights)	133.11 MiB	0.258 s	1.082 s	0.528 s	2.011 s / 1.457 s	7.27 s / 6.71 s	6.1% / 5.6%	1.695 s	3.96 s / 3.41 s	100% Match (cce52f48)
256 MB (Weights)	261.13 MiB	0.375 s	5.796 s	1.473 s (3.9×)	7.481 s / 3.158 s	12.86 s / 8.53 s	10.7% / 7.1%	2.108 s	9.96 s / 5.64 s	100% Match (4b1b1359)
512 MB (Weights)	517.15 MiB	0.760 s	16.120 s	4.210 s (3.8×)	19.387 s / 7.477 s	25.15 s / 13.24 s	21.0% / 11.0%	7.807 s	27.95 s / 16.04 s	100% Match (ab028a06)
1024 MB (1 GB)	1029.21 MiB	7.464 s	32.060 s	13.743 s (2.3×)	42.630 s / 24.313 s	55.09 s / 36.78 s	45.9% / 30.6%	17.328 s	67.42 s / 49.11 s	100% Match (c0da7edf)

1) Bit-Exact Restoration within Spot SLA: In our exploratory single-trial scaling ($N = 1$ per size), all heaps up to 1024 MB restored with 100% SHA-256 bit-level integrity. Under worst-case Spot detection ($T^{\text{worst}}_{\text{detect}} = 5.0$ s) and default gzip, total source evacuation took 55.09 s (45.9% of the 120 s Spot SLA). However, extrapolating default gzip to 2 GB risks exceeding the 120 s preemption deadline due to non-linear writeback growth, demonstrating why uncompressed packaging or iterative pre-copying is required.
2) EBS gp3 Storage Throughput and Writeback in T_{freeze}: From 26 MB to 512 MB, T_{freeze} remains sub-second (144 ms → 760 ms) as writes are buffered in page cache. At 1 GB, uncommitted dirty memory in /proc/meminfo hits 783.6 MB (vm.dirty_background_ratio = 10), triggering kernel flusher threads to serialize writes to disk. The observed write throughput of 1029.2 MiB/7.464 s ≈ 137.9 MiB/s (144.6 MB/s) is consistent with AWS EBS gp3 baseline storage throttling (125 MiB/s [18]) combined with concurrent page caching and multi-tenant memory pressure from co-located services.
3) Gzip Incompressibility and –disable-compression Speedup: Incompressible floating-point weights produce OCI layers matching the heap size. Under default Buildah single-threaded gzip, T_{build} consumes 32.060 s at 1 GB. Passing –disable-compression (-D) bypasses gzip serialization, slashing 1 GB packaging to 13.743 s (a 2.3× speedup, saving 18.32 s) and reducing worst-case evacuation from 55.09 s to 36.78 s (30.6% of Spot budget). For cluster-local registries over local VPC links (where 1029 MiB pulls in 8.5 s, matching the ≈ 1 Gbps baseline network tier), disabling compression is optimal.

D. Why Migrate? Checkpoint Restoration vs. Cold State Rebuild

A standard cold reschedule launches an empty container in ≈ 1.0 s, but resets in-memory application state to zero. As an analytical baseline model for comparison, downloading a 1 GB snapshot from cloud object storage (Amazon S3) over assumed multi-tenant bandwidth (300–800 Mbps) requires an estimated 10.2–27.3 s transfer plus 1.05–2.5 s deserialization, yielding a recovery window of 11.2–29.8 s (or minutes if recomputed).

At 1 GB, migration cutover ($T_{\text{restore}} = 17.328$ s) is comparable to this cold reconstruction estimate, while incurring 24.3 s state drift (RPO) and 36.8 s worst-case evacuation under decoupled mode with uncompressed packaging (-D), compared to 42.6 s drift and 55.1 s evacuation under gzip; or an analytical stop-and-copy projection of 49.1 s client blackout under -D versus 67.4 s under gzip. For smaller heaps ($\leq$ 256 MB), cutover is 0.95–2.10 s, trading off 0.56–3.16 s state drift under -D (0.67–7.48 s under gzip), or an analytical stop-and-copy blackout projection of 1.77–5.64 s under -D (1.88–9.96 s under gzip). Target restoration decomposes into four phases: (1) layer pull over VPC (≈ 8.5 s, matching ≈ 1 Gbps baseline), (2) CRI-O storage extraction (≈ 4.8 s), (3) runc/criu restore memory mapping and unfreezing (≈ 3.2 s), and (4) Kubelet PLEG synchronization (≈ 0.8 s).

Network transfer and disk unpacking account for $> 76\%$ of restoration latency.

E. Architectural Limitations of Single-Container Checkpointing with In-Pod Sidecars

To evaluate service meshes, we deployed a two-container pod (bench-sidecar-pod) with stream-server (:8090) and an envoy-sidecar (:15006) forwarding over loopback TCP. KEP-2008 dumped stream-server in 118 ms, and CRI-O restored it on k8s-worker-2. However, because KEP-2008 operates per container and CRIU closes established TCP sockets (tcp-close), the restored application was isolated from its sidecar proxy ($T_{\text{blackout}} = \infty$, timing out at 120 s). While future APIs could snapshot full pod network namespaces, node-level routing (Cilium eBPF host routing) decouples proxy lifecycles, offering a superior production architecture.

V. Related Work

Clark et al. [10] established VM live migration via iterative pre-copy. Voyager [7] and Stoyanov and Kollingbaum [6] combined CRIU with layered storage. Reber [8] standardized KEP-2008 checkpointing in Kubernetes. Ma et al. [15] analyzed layered container storage, and Puliafito et al. [16] evaluated migration in fog testbeds. Recent systems address operational edge and cloud migration: Zhang et al. [11] introduced KubeSPT; Calagna et al. [12] modeled edge migration; Bellavista et al. [13] evaluated ML migration; and Poggiani et al. [14] studied multi-cluster migration with Liqo and CRIU. Complementing federation frameworks, our study provides the first AWS EC2 bottleneck analysis quantifying upstream KEP-2008 and CRI-O bottlenecks: Kubelet PLEG jitter, EBS storage throughput limits, and uncompressed OCI packaging scaling up to 1 GB.

VI. Limitations, Threats to Validity, and Security

Our workload employs HTTP/1.0 with CRIU's tcp-close; persistent connections require ingress buffering or –tcp-established. Multi-gigabyte heaps would benefit from direct criu page-server streaming. Threats to Validity: We acknowledge key limitations: (1) Workload realism: monotonic counters and static arrays exhibit minimal dirtying; write-heavy workloads increase dirtying rates ($\dot{M}_{\text{dirty}}$) and drift. (2) Sample size: heap scaling is exploratory ($N = 1$); multi-trial testing on tmpfs and provisioned gp3 is required to isolate writeback dynamics. (3) Residual delay: isolating EndpointSlice from PLEG requires dual probing (runtime unfreeze, direct Pod IP probing, endpointslice -w logging, and external ClusterIP probing). (4) S3 recovery and Mode B are analytical projections. Security Considerations: Memory dumps contain plaintext secrets. In our prototype, checkpoint images are pushed to an unauthenticated cluster registry, and the operator runs as a privileged DaemonSet utilizing nsenter. Production deployment necessitates: (1) cryptographic checkpoint encryption at rest, (2) mutual TLS and signed OCI images, and (3) replacing root DaemonSets with scoped Linux capabilities (CAP_CHECKPOINT_RESTORE [19], CAP_SYS_PTRACE).

VII. Conclusion

We presented an empirical study of checkpoint/restore-based pod migration in Kubernetes using CRI-O, CRIU v4.3, Buildah, and Cilium eBPF. Across $N = 36$ migrations and $N = 20$ cold reschedules on AWS EC2, cutover blackout showed an overhead of 45.5 ms in median (+4.5%) and 125.4 ms in mean (+12.5%, 95% CI $[-6.7, +257.4]$ ms; Hodges-Lehmann diff: 136.0 ms; $p = 0.0660$). Variance is governed by Kubelet's 1 s PLEG relist cycle rather than process memory instantiation, with direct CRI-O daemon profiling confirming container unfreezing at 478–516 ms, while residual cutover latency is consistent with Kubelet status reporting and downstream control-plane convergence. In exploratory scaling up to 1024 MB, uncompressed Buildah packaging slashes build time from 32.06 s to 13.74 s (2.3×, completing evacuation in 36.78 s within the 120 s Spot deadline) with 100% SHA-256 integrity, while writeback throughput (137.9 MiB/s) aligns with EBS gp3 baseline storage limits at 1 GB. Checkpointing eliminates external persistence tiers, trading bounded state drift (0.56–3.16 s under uncompressed -D vs. 0.67–7.48 s under gzip) for 1.77–5.64 s stop-and-copy cutover at ≤ 256 MB under -D (1.88–9.96 s under gzip). Finally, single-container checkpointing severs loopback sidecars, favoring node-level routing.

Artifact Availability

Manifests, playbooks, operator code, and benchmarks are open-source: https://github.com/musaxasmammedov7/final-research.

References

[1] B. Burns et al., "Borg, Omega, and Kubernetes," CACM, vol. 59, no. 5, pp. 50–57, 2016.
[2] Netflix TechBlog, "Creating Your Own EC2 Spot Market," Netflix Inc., 2015.
[3] AWS, "Spot Instance Interruptions and EC2 Rebalance Recommendations," 2024.
[4] Redis Ltd., "Redis Latency Problems Troubleshooting," https://redis.io, 2024.
[5] CRIU Authors, "CRIU CLI Options and Architecture," https://criu.org, 2024.
[6] R. Stoyanov and M. Kollingbaum, "Efficient Live Migration of Linux Containers," in Proc. ISC Workshops, Springer, 2018, pp. 184–193.
[7] S. Nadgowda et al., "Voyager: Complete Container State Migration," in Proc. IEEE ICDCS, 2017, pp. 2137–2142.
[8] A. Reber, "Forensic Container Checkpointing in Kubernetes," Kubernetes Blog, Dec. 2022.
[9] Cilium Authors, "Cilium: eBPF Networking and Service Mesh," Linux Foundation, 2024.
[10] C. Clark et al., "Live Migration of Virtual Machines," in Proc. USENIX NSDI, 2005, pp. 273–286.
[11] H. Zhang et al., "KubeSPT: Stateful Pod Teleportation for Resilience With Live Migration," IEEE Trans. Serv. Comput., vol. 18, no. 3, pp. 1500–1514, 2025.
[12] A. Calagna et al., "Design, Modeling, and Implementation of Robust Migration of Stateful Edge Microservices," IEEE TNSM, vol. 21, no. 2, pp. 1877–1893, 2024.
[13] P. Bellavista et al., "Kubernetes Enhanced Stateful Service Migration for ML Applications," in Proc. IEEE AIoT, 2024.
[14] L. Poggiani et al., "Live Migration of Multi-Container Pods in Serverless Edge Systems," in Proc. ACM SEATED, 2024.
[15] L. Ma, S. Yi, N. Carter, and Q. Li, "Efficient Live Migration of Edge Services Leveraging Container Layered Storage," IEEE Trans. Mob. Comput., vol. 18, no. 9, pp. 2020–2033, 2019. DOI: 10.1109/TMC.2018.2871842.
[16] C. Puliafito et al., "Container Migration in the Fog: A Performance Evaluation," Sensors, vol. 19, no. 7, p. 1488, 2019. DOI: 10.3390/s19071488.
[17] P. Sharma, T. Guo, X. He, D. Irwin, and P. Shenoy, "Flint: Batch-Interactive Data-Intensive Processing on Transient Servers," in Proc. 11th ACM EuroSys, 2016.
[18] AWS, "Amazon EBS Volume Types: General Purpose SSD Volumes (gp3)," AWS Documentation, 2024.
[19] Linux Kernel Org., "capabilities(7) — CAP_CHECKPOINT_RESTORE," 2024.